

UNIVERSIDAD NACIONAL DE PILAR
**TECNICATURA UNIVERSITARIA
EN DESARROLLO DE SOFTWARE**

WEB

DESARROLLO FULL STACK

[2 0 2 6]

[Explicación exhaustiva del index.html del Target Analyzer.]

Módulo de Configuración (El Cerebro Invisible)

Línea 1: `<!DOCTYPE html>`

El `<!DOCTYPE>` es una declaración técnica y obligatoria y **debe ser la línea número 1**.

No es una etiqueta HTML propiamente dicha porque no genera ningún contenido, no se cierra y no crea un elemento en el árbol de la página (DOM) -
Banquemos develar el concepto de DOM para la próxima clase-

Es una orden directa al motor del navegador para avisarle qué versión del lenguaje estamos usando. **Estamos en el estándar HTML5**. Sin esto, como ya hemos visto antes, el navegador entra en un modo “*antiguo*” y podría *romper* nuestro diseño.

Línea 2: `<html lang="es">`

La etiqueta `<html>` es el contenedor raíz; **envuelve todo el universo de nuestro proyecto**. Aquí aparece nuestro primer **atributo**, que es `lang` (language), con el **valor** “es”. Esto indica a los motores de búsqueda -y a los lectores de pantalla para no videntes- que nuestro contenido está en español.

Línea 3: `<head>`

Abre la cabecera del documento. Todo lo que colocamos dentro de la etiqueta `<head>` es **metadatos** (*datos sobre datos*).

Un concepto sobre **DIKW** al final del párrafo

Es la configuración técnica que el usuario no ve directamente en su monitor (o pantalla), pero que es indispensable para que la página funcione, se indexe en buscadores y se vea bien.

El `<head>` es el cerebro invisible del código,
mientras que el `<body>` es el cuerpo visible.

Línea 4: <meta charset="UTF-8">

La etiqueta <meta> **provee metadatos** (datos sobre los datos).

Se encuentra exclusivamente en el <head>.

Funcionan de la forma "**Atributo = Valor**" o sea que el atributo es variable y el valor instancia en ese momento el atributo con un valor determinado.

En el caso de la etiqueta de **idioma (charset)** usa **una sola pareja** de Atributo = Valor.

En **todos los demás casos** las demás etiquetas meta (como viewport o description) usan **dos parejas** de "Atributo = Valor" (**name** y **content**) en la misma línea para poder dar una orden completa.

El atributo charset define la codificación de caracteres. El valor "UTF-8" es el estándar global que garantiza que nuestra web pueda procesar tildes, eñes y símbolos especiales sin escupir cuadrados rotos en la pantalla.

Línea 5: <meta name="viewport" content="width=device-width, initial-scale=1.0">

La primera pareja de la etiqueta identifica el objetivo: le dice al navegador **qué es** lo que estamos configurando...

name="viewport" Así se le avisa al navegador "Instancio el atributo name con viewport, entonces, estamos hablando de la pantalla, de la ventana gráfica (*viewport* en inglés significa *ventana gráfica* en castellano), del área visible del usuario.

La segunda pareja indica y el atributo content le dice **cómo** lo estamos configurando:

content="width=device-width, initial-scale=1.0"

Donde:

width=device-width:

Ordena al navegador ajustar el ancho de la página web para que coincida exactamente con el ancho de la pantalla del dispositivo en píxeles lógicos o CSS.

Sin esto, los móviles simulan una pantalla de escritorio (normalmente 980px), haciendo que todo el texto y los botones se vean mínimos.

initial-scale=1.0: Establece el nivel de zoom inicial al cargar la página por primera vez en un valor de 1:1 (100%). Esto evita que el contenido aparezca ampliado o alejado automáticamente y asegura transiciones correctas al girar la pantalla entre modo vertical y horizontal.

12 ..8015 ..36

Un comentario sobre DPR.

Línea 6: <title>CTU | Target Analyzer</title>

La etiqueta <title> define la identidad del proyecto. El texto que envuelve es exactamente lo que va a leer el usuario en la pestaña superior de su navegador Chrome, Firefox o Edge. Si el texto es muy largo, el navegador lo va a cortar, por lo que las palabras clave más importantes siempre deben ir al principio.

Es la etiqueta más importante del <head> para conectar el código con el mundo real según el MDN Web Docs.

<title>: Es el título azul que aparece en los resultados de búsqueda de Google

<meta name="description">: *(que no tenemos en nuestro código, pero viene al caso para completar el concepto)* Es el párrafo corto que aparece debajo del título en Google. Sirve para seducir al usuario para hacer un clic.

<title> Copeland y Los pigmeos de África: Cómo me enseñaron música **</title>**

<meta name="description" content="Soy Stewart Copeland de The Police y te cuento mi experiencia.">

Cuando buscamos algo en internet, no vemos el código CSS ni los metadatos de los sitios. Solo vemos una lista de títulos azules. **El ser humano puede decidir hacer clic en tu página o en la de tu competencia basándose exclusivamente en lo que escribimos dentro de <title>**

Línea 7: <link rel="stylesheet" href="style.css">

La etiqueta <link> establece un puente de comunicación con un archivo externo. **El atributo rel define la relación, y su valor "stylesheet" avisa que estamos importando una hoja de estilos.** El atributo href (**Hypertext Reference**) indica la ruta de ese archivo; como usamos el valor "style.css" sin barras ni letras de discos locales, estamos aplicando una **ruta relativa limpia**.

Línea 8: `</head>`

La barra diagonal / indica el cierre definitivo de nuestra cabecera.

Un truco avanzado: El Favicon

Para que la pestaña de nuestro navegador se vea realmente completa, el `<title>` se acompaña de otra etiqueta de infraestructura llamada `<link>` para poner un favicon (el logo chiquito al lado del texto):

```
<link rel="icon" href="tufotoparafavicon.png" type="image/png">
```

Comentario sobre **MDN Web Docs (Mozilla Developers Network)**

Módulo Visual (El Tablero de Operaciones)

Línea 9: `<body>`

Abre el cuerpo del documento. A partir de acá, **cada línea de código tiene un impacto físico directo en los píxeles de la pantalla del usuario.**

Línea 10: `<div class="bunker-container">`

La etiqueta `<div>` crea una caja contenedora genérica, un bloque invisible que se adueña de todo el ancho horizontal de la pantalla. Su función es dar un salto de renglón inmediato (sin dejar líneas en blanco) para que todo lo que metamos adentro empiece de forma limpia en una nueva fila.

El atributo **class** es un identificador que se utiliza para agrupar varios elementos y aplicarles el mismo estilo (con CSS) o el mismo comportamiento (con JavaScript).

Específicamente en nuestro caso, el atributo `class` sirve para pegarle una etiqueta identificatoria que luego usaremos en CSS.

El valor `"bunker-container"` **es el nombre específico que elegimos e indica que esta caja maestra va a tener características especiales de diseño y que va a**

encerrar a todos los demás componentes para que no se desparramen por el monitor (Por ej.: En la stylesheet le vamos a poner a esta clase un máximo de 1200 píxeles, eso va a hacer que la caja **se plante en ese ancho** y no se desparrame más, sin importar lo grande -en píxeles- que sea el monitor del usuario). En la jerga del desarrollo web, a esta caja maestra que encierra a los demás se la conoce técnicamente como un **Wrapper** (envoltorio).

Línea 11: `<header class="consola-input">`

Esta línea define **la sección de cabecera de la página** y le asigna una clase específica llamada "consola-input"

Desglosemos y detallemos:

El búnker es sólo un envoltorio.

El `<div class="bunker-container">` es el contenedor principal. Para el navegador o para Google, este div es transparente a nivel de significado. Es solo una jaula física que nos sirve para alojar controladamente información, no aporta contenido, solo aporta límites.

El `<header>` es la primera pieza semántica o el primer contenido real.

El navegador ignora la jaula del búnker y mira qué hay adentro, al ver que el `<header>` encierra la barra de herramientas, el input y el botón y que está ubicado justo por encima del `<main>`, dictamina que este componente es la cabecera global del sitio.

La analogía del marco y el cuadro

Pensémoslo de esta manera:

El `<body>` es la **pared de la habitación**.

El bunker-container es el **marco de madera** de un cuadro.

El `<header>` es el **borde superior de la pintura** (la cabecera del cuadro).

Aunque el marco de madera esté por fuera de la pintura, el borde superior sigue siendo el encabezado de toda la obra de arte, y no de un detalle de una esquina.

Por eso, aunque esté metido adentro del búnker, este `<header>` sigue cumpliendo la función de **encabezar la aplicación web completa**.

Línea 12: `<span class="prompt-text">[TARGET_URL]:</span>`

span: Es la etiqueta de apertura. Como ya vimos, es un elemento **en línea (inline)**. No genera un salto de renglón ("ENTER") antes ni después, y ocupa únicamente el ancho exacto del texto que lleva adentro. Esto es clave porque permite que el cuadro de texto (`<input>`) que viene justo después en el código se acomode exactamente a su lado en la misma fila. **Es como un `<div>` pero sin el ENTER :-)**

class="prompt-text": Es la pareja de Atributo (class) y Valor ("prompt-text"). Le asigna un nombre de grupo a este fragmento de texto para que podamos aplicarle estilos con el CSS.

[TARGET_URL]: Es el texto plano que el usuario verá en la pantalla. En este MVP, al igual que con el botón, el uso de corchetes simula el aspecto de los sistemas operativos de consola clásicos o herramientas automáticas de escaneo de terminal de los años 80/90.

Línea 13: `<input type="text" id="target-url" placeholder="Ingrese dominio objetivo ..." autocomplete="off">`

Esta es la instrucción más compleja e interactiva. La etiqueta **`<input>` crea un campo de entrada de datos** y no necesita etiqueta de cierre (no necesita un `</input>` al final) y **se comporta como un elemento in line**. Esto permite que se acomode perfectamente a la derecha del `<span>` anterior, manteniendo todo en un único renglón fluido.

type="text": Define el comportamiento del campo [MDN Web Docs]. Le avisa al navegador que debe abrir un **textbox** que acepte letras, números y caracteres comunes (como por ej.: puntos y barras de una URL)

id="target-url": Es el **identificador único absoluto** de esta caja. Nos va a servir en JavaScript para capturar lo que el usuario escriba y poder enviarlo al motor de escaneo.

placeholder="...": Define el texto de guía en gris claro que aparece adentro del textbox cuando está vacía. En cuanto el usuario hace clic y escribe la primera letra, este texto desaparece automáticamente.

autocomplete="off": Desactiva el historial de escritura del navegador. Evita que el programa intente sugerir ingresos anteriores guardados, manteniendo la estética limpia de una terminal real.

Línea 14: `<button id="btn-scan">[INICIAR_ESCANEEO]</button>`

<button> y </button>: Definen un elemento (botón) interactivo en el que el usuario puede hacer clic o touch. Este va a ser el **gatillo** que va a disparar la ejecución del robot en nuestra fase de programación lógica.

id="btn-scan": Es el identificador único de este botón. Nos va a servir en **JavaScript** para **escuchar el clic** o en **CSS** para darle estilos específicos.

[INICIAR_ESCANEEO]: Es el texto que va a ver el usuario dentro del botón. Los corchetes suelen usarse como un recordatorio para indicar que ese texto se va a reemplazar dinámicamente con JavaScript.

Línea 15: `</header>`

Cerramos el módulo de la consola superior.

Módulo Analítico (Los Paneles)

Línea 16: `<main class="grid-paneles">`

<main>: Abrimos la caja principal que nos guarda el contenido más importante de toda la página. Esta etiqueta nos sirve para tomar automáticamente el 100% del ancho del monitor, avisándole a Google que ahí adentro arranca la esencia de nuestra aplicación.

class: Ponemos el atributo de vinculación a la izquierda del signo igual para ponerle un nombre de grupo a esta mega caja. Nos sirve para avisarle al navegador que vamos a usar ese nombre para darle las órdenes de diseño desde nuestro archivo de estilos.

= "grid-paneles": Asignamos el nombre exacto de la clase entre comillas a la derecha del signo igual. Este valor lo elegimos para activar un organizador invisible en el archivo CSS que nos va a acomodar los cuatro paneles en dos columnas en las notebooks -o uno debajo del otro en los celulares-.

Línea 17: `<section class="panel" id="panel-vista">`

La etiqueta semántica `<section>` divide el contenido en un bloque temático independiente.

Usamos **`<section>`** porque es la única etiqueta que le aporta **significado temático** a una parte de la página.

Si miramos las etiquetas que veníamos usando antes, cada una tiene una limitación o una función muy específica que no sirve para armar los cuatro paneles:

`<body>`: Es el cuerpo entero de la web. Solo puede haber uno y envuelve absolutamente todo. No nos sirve para dividir subapartados.

`<main>`: Define el centro de operaciones de la página. Al igual que el cuerpo, es único por documento. No podemos repetir cuatro `<main>` para los cuatro paneles.

`<header>`: Es exclusivamente para la cabecera, introducciones o barras de comandos. Los paneles son zonas de visualización de datos, no son encabezados.

`<div>`: Esta es la clave. El `<div>` es un contenedor totalmente mudo y genérico. No significa nada. Si usáramos cuatro `<div>` para los paneles, el navegador y Google verían cuatro cajas vacías de significado, como si tiráramos carpetas sin nombre en un escritorio.

La característica única de la `<section>`

La gran diferencia es que la `<section>` tiene **valor semántico de organización**. Le avisa al sistema operativo, a los motores de búsqueda y a los lectores de pantalla: *"Ojo, esta caja no es un simple adorno visual; es un capítulo o una sección con un tema propio que tiene su propio título".*

En nuestra interfaz, cada panel es una unidad de información independiente. Al usar `<section>`, estamos declarando de forma limpia y ordenada que el "Panel 1" habla de un tema, el "Panel 2" de otro, y que cada uno funciona como un módulo autónomo dentro de la aplicación.

Acá combinamos dos identificadores: la class con valor "panel" (que se va a repetir en los cuatro cuadrantes para darles el mismo color y borde de diseño) y el id con valor "panel-vista" (que es el PANEL 1 y es el único para inyectar allí las imágenes del robot).

Línea 18: <h2 class="panel-titulo">PANEL 1: [VISTA]</h2>

La etiqueta <h2> representa un título de segundo nivel, ideal para subtitular las secciones dentro de nuestro búnker.

Línea 19: <div class="contenido-panel">Esperando objetivo ... </div>

Un contenedor genérico div con la clase "contenido-panel". Es la caja vacía donde impactarán los datos dinámicos.

Línea 20: </section>

Cerramos el primer cuadrante.

Líneas 21 a 24:

Se repite con exactitud la anatomía estructural semántica para el **Panel 2**. La única variante técnica es que el atributo id toma el valor único "panel-tech" para prepararse a recibir la información de los servidores.

Líneas 25 a 28:

Se estructura el **Panel 3. Detalle crítico en la Línea 27:** <div class="contenido-panel scroll-activo">. Aquí dentro del atributo class pasamos **dos valores separados por un espacio**. Esto significa que esa caja hereda el diseño básico de "contenido-panel", pero además le sumamos la lógica de "scroll-activo" para contener el futuro desbordamiento de (presumiblemente) cientos de enlaces.

Líneas 29 a 32:

Se repite la estructura semántica final para el **Panel 4**, utilizando el id específico "panel-metricas".

Cierre del Sistema

Línea 33: </main>

Cerramos el núcleo de la grilla de paneles.

Línea 34: </div>

Cerramos la caja maestra que definimos en la línea 10 (bunker-container).

Línea 35: </body>

Cerramos el cuerpo visual del documento.

Línea 36: </html>

Cerramos el código HTML completo. Fin del esqueleto operativo.